

SECURITY REVIEW

FOR MYSTIC FINANCE



mystic[★]

INSIDE THIS REPORT

CONTENTS

01	AUDIT SUMMARY	03
02	SCOPE	04
03	AUDITORS	05
04	METHODOLOGY	06
05	RISK CLASSIFICATION	07
06	FINDINGS SUMMARY	08
07	DETAILED FINDINGS	09
	C-01 Miscalculation in rewards will give too much rewards to users	09
	M-01 DoS Vulnerability in withdraw() Due to Overcommitment of currentWithheldETH in Concurrent unstake() Calls	11
08	DISCLAIMER	12
09	ABOUT KANN AUDITS	13

01 - OVERVIEW

AUDIT SUMMARY

Mystic Finance engaged Kann Audits to perform a security review of the Mystic Finance smart contracts. A team of 1 security researchers performed a manual review of the in-scope contracts. All findings identified during the assessment are documented in this report.

TOTAL FINDINGS

2

 ISSUES

● CRITICAL	01
● HIGH	-
● MEDIUM	01
● LOW	-
● INFORMATIONAL	-

ENGAGEMENT DETAILS

PREPARED FOR	Mystic Finance
PREPARED BY	Kann Audits
DATE REPORTED	12 June 2025
METHODS	Manual review

02 - WHAT WE REVIEWED

SCOPE

REPOSITORY	https://github.com/mystic-finance/Liquid-Staking/tree/staked-plume
AUDITED COMMIT	2e6d7e59def340c63f98808b19c9650d8f63917b

#	FILE
01	/src/stPlumeMinter.sol
02	/src/frxETH.sol

03 - THE TEAM ON THIS ENGAGEMENT

AUDITORS



Kann

LEAD SECURITY RESEARCHER

METHODOLOGY

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

STEP BY STEP

OUR SECURITY PROCESS

01

Kickoff & Context

A kickoff call to understand architecture, trust boundaries, threat assumptions and intended behaviour.

02

Scope & Preparation

Confirm the commit, files in scope, dependencies and the riskiest areas. We open a private channel between your team and our auditors.

03

AI First Scan

Kann AI, our proprietary model built to find security bugs, scans the codebase before the manual review starts.

04

Expert Security Review

Line-by-line analysis of exploit paths, invariant breaks, edge cases, accounting and integration risks.

05

Findings & Proofs

Each issue is confirmed, rated and backed by a proof of concept or a precise reproduction path.

06

Mitigation Review

We review every fix, confirm it resolves the root cause and flag partial or incorrect mitigations.

COVERAGE

THE REVIEW SPECIFICALLY CONSIDERED

- ✓ Testing against common and uncommon attack vectors
- ✓ Contract logic matching the client's intended behaviour
- ✓ Manual line-by-line review by experienced auditors
- ✓ Architecture and design analysis for secure component interactions and clear trust boundaries
- ✓ Compliance with industry best practices and known smart contract security standards
- ✓ Cross-referencing patterns with industry-standard implementations
- ✓ Access control verification
- ✓ State transitions and logic, to detect inconsistencies, unexpected behaviour or exploitation paths

RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

● CRITICAL	Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.
● HIGH	Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
● MEDIUM	Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.
● LOW	Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.
● INFORMATIONAL	Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability.

06 - ALL ISSUES

FINDINGS SUMMARY

● CRITICAL	● HIGH	● MEDIUM	● LOW	● INFO
01	-	01	-	-
TOTAL 02 FIXED 02 ACKNOWLEDGED 00				

ID	TITLE	SEVERITY	STATUS
C-01	Miscalculation in rewards will give too much rewards to users	● CRITICAL	● FIXED
M-01	DoS Vulnerability in withdraw() Due to Overcommitment of currentWithheldETH in Concurrent unstake() Calls	● MEDIUM	● FIXED

DETAILED FINDINGS

C-01

Miscalculation in rewards will give too much rewards to users

SEVERITY

● CRITICAL

STATUS

● FIXED

■ DESCRIPTION

In `syncRewards`, the rewards are calculated by subtracting the last rewards to the total yield. This is inaccurate because total yield keep increasing, and last rewards are just the last amount rewarded.

This means that over time, rewards will become much more inflated than their actual values, giving away too many yield to the users. The first users to withdraw will effectively steal from others.

■ PROOF OF CONCEPT

Let's assume that we get 10 ether of rewards per cycle, for simplicity.

The key parts of function `syncRewards` work as follow:

Cycle 1:

`yieldEth` has increased to 10 ether. `LastRewardAmount` is 0 (first run).

```
uint256 nextRewards = yieldEth - lastRewardAmount
uint256 nextRewards = 10 ether - 0
= 10 ether
```

`LastRewardAmount` is set to `nextRewards`, 10 ether.

Cycle 2:

`yieldEth` has increased to 20 ether. `LastRewardAmount` is 10.

```
uint256 nextRewards = 20 ether - 10 ether = 10 ether
```

`LastRewardAmount` is set to `nextRewards`, 10 ether.

Cycle 3, where things start to break:

`yieldEth` has increased to 30 ether. `LastRewardAmount` is 10.

```
uint256 nextRewards = 30 ether - 10 ether = 20 ether
```

Next rewards are calculated as twice the normal amount.

■ REMEDIATION ● FIXED

M-01

DoS Vulnerability in withdraw() Due to Overcommitment of currentWithheldETH in Concurrent unstake() Calls

SEVERITY

● MEDIUM

STATUS

● FIXED

■ DESCRIPTION

The protocol maintains a global variable `currentWithheldETH` to represent the amount of ETH reserved for user withdrawals after they call `unstake()`. However, this variable is only decreased when `withdraw()` is called - not when `unstake()` is made. This creates a critical race condition: multiple users can initiate `unstake()` relying on the same `currentWithheldETH` balance, leading to overcommitment. If one user completes the withdrawal first, the remaining users are blocked from withdrawing, resulting in a denial of service (DoS).

Example Scenario Initial State: `currentWithheldETH` = 10 ETH

Alice calls `unstake(10 ETH)` → succeeds (as 10 ETH is available)

Bob calls `unstake(8 ETH)` → also succeeds (still sees 10 ETH as available)

Both transactions pass the `currentWithheldETH >= amount` check, since the variable hasn't been decremented yet.

Alice calls `withdraw()` first:

Receives 10 ETH

`currentWithheldETH` becomes 0

Bob then calls `withdraw()`:

Fails, as there is no remaining `currentWithheldETH`

Additionally, Bob cannot call `unstake()` again due to the presence of an active withdrawal request

If no one deposits or triggers a stake rebalance before Bob's cooldown period ends, Bob is temporary locked out of his funds until admin calls `unstakeGov()` or rebalance happens.

■ REMEDIATION

● FIXED

DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section at the audited commit. Code changed after that commit, external dependencies, deployment configuration and operational practices may introduce additional risk.

ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.



EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.



AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.



FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.



INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.



KANN AUDITS
WEB3 SECURITY AUDITS

GET IN TOUCH

WEBSITE	kannaudits.com
PORTFOLIO	github.com/KannAudits
EMAIL	contact@kannaudits.com
X	@KannAudits
TELEGRAM	@KannAudits